

Emotion recognition in speech

First author
Nayeli Guzman Huayta

Second author
Andrea Coa Cruz

Third author
Luis Ponce

Fourth author
Matías Meneses Roncal

ABSTRACT

El presente documento explora distintos métodos de clasificación para audios según la emoción expresada en la voz. Se utilizan técnicas de extracción de características como MFCC y ZCR, junto con modelos de aprendizaje automático y redes neuronales.

Key words: emotion classifier, CNN, audio, XG, MFCC

1. INTRODUCCIÓN

El análisis y clasificación de emociones a partir de audios es un área clave en Machine Learning, con aplicaciones en salud mental, atención al cliente, seguridad ocupacional, entre otros. Identificar patrones emocionales en la voz permite desarrollar tecnologías más empáticas y personalizadas, mejorando la interacción entre humanos y sistemas automatizados.

La principal motivación detrás de este proyecto es la capacidad de identificar la emoción de una persona a través de su tono de voz, lo que permite inferir su estado de ánimo. Esta información puede ser utilizada en múltiples ámbitos. En atención al cliente, por ejemplo, los sistemas de respuesta de voz interactiva (**IVR**) podrían detectar si un usuario está molesto y redirigirlo automáticamente a un agente humano, optimizando la experiencia del servicio. En el caso de los asistentes virtuales, estos podrían ajustar su tono de respuesta o hacer preguntas empáticas como: "Disculpa por este inconveniente, ¿cómo crees que puedo mejorar?", mejorando la interacción con los usuarios.

Para lograr entrenar un modelo que clasifique audios por emoción, se empleará un conjunto de datos de audios etiquetados con diferentes emociones y se aplican técnicas de extracción de características como la tasa de cruces por cero (**ZCR**), la energía de la señal (**RMSE**), y los coeficientes cepstrales de frecuencia de Mel (**MFCC**). Además, se incorporan transformaciones como la adición de ruido y cambios en el tono para mejorar la robustez del modelo ante variaciones en la voz.

El entrenamiento se llevó a cabo utilizando tres enfoques:

- **SVM con reducción de dimensionalidad**, para evaluar su efectividad en la clasificación de emociones.
- **1D-CNN sin reducción de dimensionalidad**, para analizar cómo la red maneja directamente los datos en alta dimensión.
- **XGBoost sin reducción de dimensionalidad**, con el objetivo de comparar su desempeño frente a los otros modelos.

Este enfoque comparativo permite evaluar el impacto de la dimensionalidad en la precisión del modelo, optimizando el balance entre rendimiento computacional y capacidad predictiva.

2. REVISIÓN DE LITERATURA

Siendo la identificación de emociones un problema de clasificación, hay dos principales tipos de soluciones propuestas: usar métodos tradicionales de clasificación (por ejemplo: SVMs); o usar redes neuronales, mayormente presente en literatura reciente.

En literatura más antigua, SVM es utilizado como clasificador. Varios autores, como [1] en 2020, y [2] en 2012 utilizan SVMs tradicionales, ambos con features de audios relacionados al tono (*pitch*) y energía, así como MFCC y LPCC, obteniendo algunos resultados bastante buenos, por encima del 90 % de *accuracy*.

Un acercamiento distinto para la utilización de SVM es propuesto por [3] en 2014, en la que usan una variante de SVM: ranking SVM. Utilizan 6 rankers (uno por cada emoción); cada uno ordena la data según el grado en el que expresa determinada emoción. Por ejemplo: el ranker de felicidad ordena los audios desde el que expresa más felicidad hasta el que menos felicidad expresa. Finalmente se pueden aplicar distintos métodos para decidir cuál es la emoción predominante en un audio; por ejemplo, para un audio determinado, elegir la emoción en la que tuvo el mejor ranking. Con el dataset Berlin, esta propuesta llegó a más de 90 % de *accuracy*.

Por otro lado, dentro de las propuestas que involucran redes neuronales, tanto RNNs (en particular, LSTM) como CNNs han sido utilizadas.

En 2022, [4] propone una arquitectura de CNN profunda compuesta de 5 capas convolucionales y 5 capas de *max-pooling*, junto con 2 capas *fully-connected* para la clasificación. Antes de alimentar la CNN con los videos, extrae las características utilizando los métodos Zero-Crossing Rate, MFCC, Chroma STFT, Root Mean Square, Mel Scale Spectrogram. Utilizando los datasets RAVDNESS, TESS, y CREMA-D, alcanza un porcentaje de *accuracy* de 96 %, 99 % y 84 %, respectivamente.

Una propuesta un poco diferente consiste en utilizar modelos tanto para los audios como para transcripciones de estos, y finalmente juntar los resultados, creando un modelo que trabaje con audios y se complementa con los textos. Esta es la propuesta de [5]. Primero extrajeron características de los audios del dataset IEMOCAP, y de los textos respectivos a cada audio. Las características de los audios fueron procesadas por una red densa, mientras que las ca-

racterísticas de los textos fueron procesadas por un LSTM. Los resultados de ambas redes fueron ensamblados en otra red densa. Esta estrategia alcanzó 75.48 % de *accuracy*, superando a otros modelos que trabajan con el mismo dataset.

Otro enfoque, propuesto por [6], consiste combinar una red convolucional 2D de 4 bloques convolucionales y una LSTM con una capa de *self-attention*. La red convolucional recibe los features, y los outputs de esta red sirven de input para la LSTM. Finalmente usa una red densa para obtener las predicciones. Los features de los audios fueron extraídos con MFCC. La capa de atención introducida en este método ayuda a dar más o menos importancia a ciertos elementos de los inputs. El porcentaje de *accuracy* obtenido con esta combinación de redes fue de 90.19 %.

3. DATASET

3.1 Descripción del dataset

Con el fin de obtener un conjunto de datos extenso y representativo para el entrenamiento de los modelos propuestos, se ha construido un dataset a partir de la combinación de cuatro bases de datos distintas obtenidas de Kaggle. Estas han sido seleccionadas debido a su enfoque común en la clasificación de emociones:

- **RAVDESS**: Contiene 1440 archivos .wav de habla y canto emocional, clasificados en **8 emociones**: neutral, calm, happy, sad, angry, fearful, disgust, and surprised.
- **CREMA-D**: Contiene 7442 archivos .wav clasificados en **6 emociones**: anger, disgust, fear, happiness, neutral, and sadness.
- **TESS**: Contiene 2800 archivos .wav clasificados en **7 emociones**: anger, disgust, fear, happiness, pleasant surprise, sadness, and neutral.
- **SAVEE**: Contiene 480 archivos .wav grabados por cuatro actores, clasificados en **7 emociones**: neutral, anger, disgust, fear, happiness, sadness, and surprise.

Como se puede observar, no todos los datasets contienen el mismo conjunto de emociones. En particular, el dataset RAVDESS incluye una mayor cantidad de categorías emocionales, y para mantener la coherencia en la clasificación, se ha considerado que la emoción *neutral* es equivalente a *calm* dentro de este conjunto de datos. Tras esta modificación la distribución final de emociones se muestra en la Figura 1.

Actualmente, el conjunto de datos concatenado cuenta con un total de **12 162** archivos de audio.

Al analizar los espectrogramas de las emociones *Happy* 9, *Neutral* 7 y *Angry* 8, se observan diferencias significativas en la distribución de la energía en las frecuencias a lo largo del tiempo.

El espectrograma de *Happy* muestra una mayor intensidad en las frecuencias medias y altas, con transiciones suaves, reflejando un tono enérgico durante casi todo el tiempo.

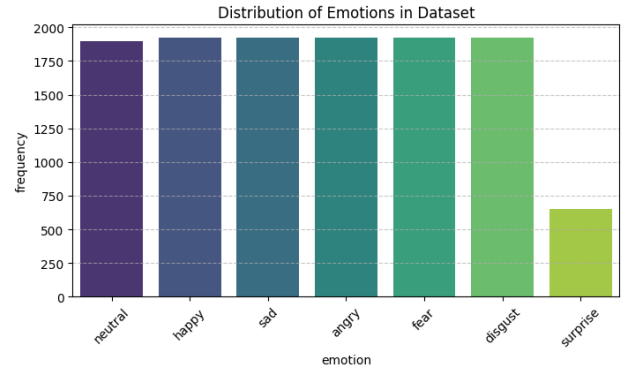


Figure 1. Distribución de emociones

que dura el audio. En contraste, *Neutral* presenta una distribución de energía más uniforme a lo largo del tiempo, con una menor presencia de frecuencias altas, lo que sugiere una entonación estable y sin grandes fluctuaciones. Por otro lado, *Angry* tiene un espectrograma con mayor energía en las frecuencias bajas y medias, con transiciones más bruscas y patrones irregulares, esto es característico de una voz con mayor tensión y agresividad.

Estas diferencias en la energía y el patrón espectral permiten identificar las emociones en el habla de manera visual.

3.2 Data augmentation

Se utilizará esta técnica para aumentar la cantidad y diversidad de datos en el conjunto de entrenamiento, lo que va a ayudar a mejorar la capacidad de generalización de los modelos propuestos. Como estamos analizando audios, se aplicarán ciertas transformaciones para simular variaciones naturales en las señales de sonido.

3.2.1 Noise — Ruido

- Se agrega ruido aleatorio a la señal de audio para simular grabaciones en entornos ruidosos.
- Esto ayuda al modelo a volverse más robusto ante variaciones en las condiciones de grabación.

3.2.2 Time Shifting — Desplazamiento Temporal

- Desplaza la señal de audio en el tiempo para generar versiones modificadas de la misma grabación.
- Ayuda a que el modelo sea más robusto ante pequeños cambios en la alineación temporal del audio.

3.2.3 Pitch Shifting — Cambio de Tono

- Altera la frecuencia de la señal de audio sin cambiar su duración.
- Simula variaciones en las voces de los hablantes.

Por cada audio que se cargue con *librosa*, se generarán nuevos audios a partir de ese, uno con ruido, *shift* y otro con ruido y *pitch* a la vez. De esta manera, por cada audio se generan 3 más.

3.3 Audio feature extraction

Cada uno de los audios generados pasará por un proceso de extracción de características, por lo tanto, es fundamental tener un buen proceso de extracción que permita representar información contenida en la señal de cada audio. En el presente proyecto se presentan tres técnicas de extracción de características utilizadas en el procesamiento de señales de audio:

3.3.1 Zero Crossing Rate (ZCR)

La tasa de cruces por cero (ZCR) representa la cantidad de veces que la señal de audio cruza el eje horizontal en un intervalo de tiempo y se define como:

$$ZCR_t = \frac{1}{2N} \sum_{n=1}^N |\text{sgn}(x(n)) - \text{sgn}(x(n-1))| \quad (1)$$

donde N es el número de muestras en la ventana de análisis, $x(n)$ es la señal de audio en la muestra n , y la función signo se define como:

$$\text{sgn}(x(n)) = \begin{cases} 1, & \text{si } x(n) > 0 \\ 0, & \text{si } x(n) \leq 0 \end{cases} \quad (2)$$

3.3.2 Root Mean Square Energy (RMSE)

La energía cuadrática media (RMSE) mide la intensidad de la señal de audio y se calcula como:

$$RMSE_t = \sqrt{\frac{1}{N} \sum_{n=1}^N x(n)^2} \quad (3)$$

donde $x(n)$ representa la amplitud de la señal en la muestra n , y N es el número total de muestras en la ventana de análisis.

3.3.3 Mel Frequency Cepstral Coefficients (MFCC)

Los coeficientes cepstrales en la escala Mel (MFCC) representan la envolvente espectral de la señal de audio mediante una transformación no lineal de la escala de frecuencia. Los pasos para su cálculo son los siguientes:

1. **Segmentación:** Las señales de audio se dividen en *frames* utilizando un tamaño de ventana y un desplazamiento fijo
2. **Transformada Rápida de Fourier (FFT):** Aplicar la FFT a cada *frame* para obtener su representación en el dominio de la frecuencia.
3. **Escala Mel:** Se aplican filtros basados en la escala Mel, que imita la percepción auditiva humana de las frecuencias.
4. **Compresión logarítmica:** Calcular el logaritmo de las magnitudes de los coeficientes filtrados para imitar la sensibilidad logarítmica del oído humano a la intensidad del sonido.

5. **Transformada Discreta del Coseno (DCT):** Aplicar la DCT para reducir la redundancia y obtener un conjunto compacto de coeficientes representativos de la envolvente espectral de la señal.

Estas técnicas permiten extraer características significativas de una señal de audio, facilitando su análisis en aplicaciones de reconocimiento de emociones, clasificación de audio y procesamiento del habla.

Como se mencionó al inicio, los audios provienen de distintos datasets, lo que significa que sus duraciones varían, esto resulta en una diferente cantidad de características extraídas para cada uno. Para garantizar uniformidad, los audios con menor cantidad de características se rellenarán con ceros hasta igualar la cantidad de características del audio más largo, teniendo así audios de 2376 dimensiones.

4. METODOLOGÍA

4.1 Métodos de reducción de dimensionalidad

Con propósitos experimentales, elegimos un método de reducción basado en proyecciones. Así, el método de reducción de dimensionalidad elegido es **LDA**. Se eligió LDA para aprovechar el hecho de que estamos tratando con un problema de clasificación y contamos con las etiquetas, lo cual es beneficioso para LDA porque es un método supervisado que maximiza la separación entre clases.

4.2 Modelos de clasificación

Los modelos elegidos fueron SVM, árboles de decisión (este último en la forma de *Extreme Gradient Boost*), debido a su capacidad para formar límites no lineales entre clases y CNN.

4.2.1 SVM

SVM, o Máquina de Soporte Vectorial [7], es un algoritmo de clasificación que busca un separador óptimo entre clases; es decir, que maximice la distancia entre dicho separador y los puntos de la data más próxima a él. Supongamos que tenemos un conjunto de datos $\{x_n\}$, cada observación con una etiqueta $t_n \in \{-1, 1\}$. En un planteamiento de SVM donde el separador es lineal y la ecuación de este separador es $L: y(x) = w^\top x + w_0$, la distancia entre algún punto x_n y el hiperplano L está dada por $d = \frac{t_n \cdot y(x_n)}{\|w\|}$.

Buscamos hallar la ecuación del hiperplano L que maximice la distancia entre el dicho L y punto más próximo a él; es decir, hallar los parámetros w y w_0 ,

$$\arg \max_{w, w_0} \left\{ \frac{1}{\|w\|} \min_n \left\{ t_n \cdot y(x_n) \right\} \right\} \quad (4)$$

Por conveniencia, podemos deliberadamente determinar que la distancia entre el punto más cercano al hiperplano debe ser 1, de modo que para este punto x^*

$$t_n \cdot y(x^*) = 1$$

Reemplazando esto en 4, nos quedamos con simplemente:

$$\arg \max_{w, w_0} \left\{ \frac{1}{\|w\|} \right\}$$

Lo cual podemos replantear como:

$$\arg \min_{w, w_0} \left\{ \frac{1}{2} \|w\|^2 \right\} \quad (5)$$

Esto está sujeto a las restricciones (por cada punto):

$$g_i(w, w_0) = t_n \cdot (w^\top x_i + w_0) \geq 1 \quad (6)$$

Juntas, 5 y 6 forman el problema *primal* de SVM. Es un problema de optimización con restricciones. El lagrangiano que queremos maximizar es:

$$\mathcal{L}(w, w_0, \lambda) = \frac{1}{2} \|w\|^2 - \sum_{n=1}^N \lambda_n \cdot \{t_n \cdot (w^\top x_n + w_0) - 1\}$$

donde λ es el vector con multiplicadores de Lagrange. derivando $\mathcal{L}$ con respecto a w y w_0 e igualando a 0 se obtiene que

$$w = \sum_{n=1}^M \lambda_n \cdot t_n \cdot x_n, \text{ y } \sum_{n=1}^N t_n \cdot \lambda_n = 0$$

Con estas ecuaciones es posible eliminar w y w_0 de $\mathcal{L}$, quedando así sólo una función de λ . Este planteamiento de SVM es el problema *dual*, y nos permite utilizar el *kernel trick*, el cual transforma los datos x a otro espacio. Esto es útil si, por ejemplo, las clases no son linealmente separables. Reescribiendo la ecuación anterior, buscamos maximizar:

$$\hat{\mathcal{L}}(\lambda) = \sum_{n=1}^N \lambda_n - \sum_{n=1}^N \sum_{m=1}^N \lambda_n \lambda_m t_n t_m k(x_n, x_m)$$

donde

$$k(x_n, x_m) = \phi(x_n)^\top \phi(x_m)$$

ϕ es una función que mapea x a otro espacio dimensional. Finalmente, la función $\hat{\mathcal{L}}$ está sujeta a que cada $\lambda_i \geq 0$ y $\sum_{n=1}^N \lambda_n t_n = 0$. Nos ha quedado un problema de programación cuadrática.

Justificación

El *kernel trick* permite que SVM sea más que sólo un separador lineal, que es lo que buscamos en la experimentación. Asimismo, varios autores utilizan SVM como método de clasificación en el problema de reconocimiento de emociones; sin embargo, en la literatura revisada ninguno propone utilizar SVM junto con reducción de dimensionalidad, que es lo que se plantea en el presente proyecto.

4.2.2 XGBoost

Extreme Gradient Boost es un método de clasificación y es una optimización de *Gradient Boost*. *Gradient Boost* es un *ensemble model* de árboles de decisión, en el que los árboles son entrenados secuencialmente, cada uno tratando de minimizar el error del árbol anterior.

Sea $\{x^{(n)}\}$ el conjunto de observaciones. El modelo empieza computando una probabilidad inicial, que será la predicción inicial para todas las observaciones de la data. Llamemos a esta predicción inicial F_0 . Evidentemente, tener una única probabilidad para todas las clases no sirve para clasificar. Calculamos la función de pérdida, o los residuales r : la diferencia entre el valor real y y el valor predicho F_0 . Reducir este error es la meta de los siguientes árboles del modelo.

$$r^{(n)} = y^{(n)} - F_0^{(n)}$$

El siguiente árbol del modelo es entrenado, y se calcula el *output value* h_1 de cada una de sus hojas L :

$$h_1 = \frac{\sum_{i \in L} r^{(i)}}{\sum_{i \in L} F_0^{(i)} \times (1 - F_0^{(i)}) + \lambda},$$

donde λ es un término de regularización. Cabe mencionar que en el caso multiclase, *XGBoost* se entrena utilizando la estrategia *One vs Rest*, transformando así la clasificación multiclase en una clasificación binaria. Luego utiliza alguna función como *softmax* para transformar múltiples *outputs* en probabilidades para cada clase. Es por eso que trabajamos con una única probabilidad $F_0^{(i)}$ para cada $x^{(i)}$ en lugar de un vector de probabilidades.

Finalmente la nueva probabilidad para un $x^{(n)}$ se calcula así:

$$\sigma(\text{logit}(F_0^{(i)}) + \eta \times h_1^{(i)}),$$

donde η es la tasa de aprendizaje y σ es una función que transforma el input en probabilidades. Con esta nueva predicción (que ahora sí es distinta para cada observación de la data) se puede recalcular los residuales r , y repetir el proceso hasta alcanzar el límite de estimadores especificados por el usuario. La ecuación anterior se generaliza de la siguiente forma:

$$F_m = \sigma(\text{logit}(F_{m-1}) + \eta \times h_{m-1})$$

Justificación

Se eligió trabajar con *XGBoost* debido a la mejora significativa que presenta frente a una versión más simple: un árbol de decisión. Este es un clasificador relativamente débil, que, experimentalmente, no presentaba buenos resultados clasificando los audios. *XGBoost* es capaz de generar límites no lineales entre clases, y, además es capaz de corregir sus errores para mejorar su desempeño. Asimismo, mejoras que *XGBoost* tiene sobre *Gradient Boost* es que implementa regularización y *pruning*, por lo que es menos propenso a *overfitting*.

4.2.3 Convolutional Neural Network — 1D

Las **Redes Neuronales Convolucionales** de una dimensión son un tipo de modelo diseñado específicamente para procesar datos secuenciales aplicando filtros sobre la data de entrada para capturar patrones locales. En general, esta red aplica **capas convolucionales** que mantienen la equivarianza a la traslación, y capas de **pooling** que ayudan a reducir la dimensionalidad pero inducen a una invarianza parcial.

Dado un conjunto de datos de entrada $\mathbf{X} \in \mathbb{R}^{N \times D}$, donde N es la cantidad de tuplas y D es la cantidad de características por tupla, la red le aplica una serie de filtros para aprender patrones significativos. De esta manera, la salida del modelo se define como:

$$\hat{y} = f(\mathbf{X}; \theta)$$

Donde θ representa todos los parámetros aprendidos.

A continuación, se describirá la arquitectura de una 1D-CNN:

- **Capas Convolucionales:** Estas capas aplican filtros sobre la data de entrada para extraer características locales y patrones relevantes. La operación de convolucion en la posición i se definirá como:

$$\text{OutConv}_i = \mathbf{a} \left[\beta_i + \sum_{j=1}^D W_{ij} \cdot x_j \right] \quad (7)$$

- **Capas de Pooling:** Estas capas reducen la dimensionalidad de la salida convolucional, conservando la información más relevante y reduciendo la sensibilidad a pequeñas variaciones en la posición de los patrones detectados. Los tipos más comunes de *pooling* son:

- **Max Pooling:** Toma el valor máximo en una ventana.
- **Average Pooling:** Calcula el promedio en la ventana.

En el modelo propuesto, se empleará *Max Pooling*, ya que permite conservar las características más relevantes de los audios. Al seleccionar el valor máximo dentro de cada ventana, se resaltan los patrones clave, lo que mejora la capacidad del modelo para clasificar correctamente.

- **Capas densas:** Después de aplicar varias capas convolucionales y de *pooling*, se aplanan los datos y se conectan a capas densas (fully connected layers), que funcionan como una red neuronal tradicional.
- **Capa de salida:** La última capa genera la predicción final:

- Para clasificación binaria, usa **sigmoide**.
- Para clasificación multiclase, usa **softmax**.
- Para regresión, usa una **activación lineal**.

Dado que el objetivo del modelo es la clasificación de emociones, donde existen múltiples categorías, se opta por la **función de activación softmax** en la capa de salida

Entonces la arquitectura que se propone para la 1D-CNN consiste en cinco capas convolucionales, cuatro capas de max pooling, una capa de flattening y dos capas densas.

Todas las capas están interconectadas, de modo que las características extraídas en cada capa convolucional se transfieren como entrada a la siguiente, permitiendo una mejor representación de los datos.

Justificación

Se eligió una **1D-CNN** debido a su capacidad para **capturar patrones locales en datos secuenciales**, lo cual es fundamental en la clasificación de emociones a partir de señales de audio. A diferencia de modelos como **MLP**, que no aprovechan la estructura secuencial de los datos, o **LSTMs/GRUs**, que aunque pueden modelar dependencias a largo plazo, suelen ser más costosos computacionalmente, las 1D-CNNs logran un equilibrio entre **eficiencia** y **rendimiento**. Además, su estructura con capas de **convoluciones** y **pooling** permiten extraer características clave sin requerir una gran cantidad de parámetros, lo que mejora la generalización del modelo.

5. IMPLEMENTACIÓN

La implementación de todo el proyecto se encuentra en este [repositorio de GitHub](#)

5.1 Estandarización

Tanto con **XGBoost** como **CNN** y **SVM** tuvieron mejores resultados con la estandarización. Esto garantiza que ninguna característica tenga un impacto desproporcionado sobre las demás debido a diferencias en su escala.

La estandarización en una **1D-CNN** contribuye a un entrenamiento más estable al mantener los valores de entrada dentro de un rango controlado, evitando que sean excesivamente grandes o pequeños. Esto ayuda a disminuir problemas como la desaparición o explosión del gradiente. Además, al asegurar que los datos estén en una escala homogénea, facilita una convergencia más rápida del optimizador y mejora la eficiencia en la actualización de los pesos.

La estandarización elegida fue *Z-Score Normalization*, la cual transforma los datos de tal manera que tengan una media de 0 y una desviación estándar de 1. Se calcula con la siguiente fórmula:

$$Z = \frac{x - \mu}{\sigma}$$

Donde:

- x = Valor original de la variable.
- μ = Media de la variable.
- σ = Desviación estándar de la variable.

Con la estandarización, los datos están listos para ser procesados. Primero se reducirá las dimensiones (en caso sea necesario) y luego se aplicará los modelos de clasificación elegidos.

Nota: Se probará reducción de la dimensionalidad solo con **XGBoost**, para CNN, se dejará su formato original.

5.2 Paralelización

El entranamiento de una modelo como CNN requiere una gran cantidad de operaciones matriciales, las cuales pueden ser paralelizables. Debido a esto se realizo una optimización con una tarjeta grafica GPU Nvidia GeForce RTX 4060 con 8GB de VRAM, lo que permitio una mejora significativa en rendimiento. Inicialmente se realizo el entrenamiento en una CPU, lo que resulto en tiempos de entrenamientos excesivamente largos, para optimizar el proceso se dejó de usar las funciones de TensorFlow CPU para utilizar la biblioteca de **TensorFlow/Keras con CUDA**.

5.3 Optimizador ADAM

El optimizador ADAM (Adaptive Moment Estimation) se utilizó en el entrenamiento de la 1D-CNN debido a su capacidad para ajustar dinámicamente la tasa de aprendizaje y mejorar la estabilidad del entrenamiento. ADAM combina las ventajas de Momentum y RMSprop, lo que permite una convergencia más rápida y estable. Además, se adapta bien a diferentes escalas de datos y es especialmente útil en problemas de clasificación con data augmentation, logrando un mejor ajuste y generalización del modelo.

6. EXPERIMENTACIÓN

6.1 SVM

SVM	accuracy	precision	f1-score
kernel poly (grado 6)	0.3206	0.3901	0.3182
kernel rbf	0.4626	0.4647	0.4627

Table 1. Búsqueda de hiperparámetros de SVM tras reducir la dimensionalidad (LDA)

SVM se aplicó a la data con reducción de dimensionalidad. De la tabla se observa la notoria diferencia entre aplicar el kernel polinómico y el kernel RBF (Radial Basis Function), esto se debe a a cómo cada kernel maneja la transformación de los datos de entrada para encontrar el margen óptimo en un espacio de mayor dimensión.

El modelo con kernel polinómico de grado 6 no logró generalizar tan bien, probablemente debido a un sobreajuste en los datos de entrenamiento.

6.2 XGBoost

XGBoost se aplicó a los features extraídos sin reducir la dimensionalidad. Los hiperparámetros a buscar para XGBoost fueron el número de estimadores del modelo o cantidad de árboles, profundidad máxima, y tasa de aprendizaje. A continuación se presentan los mejores resultados:

De la tabla se observa que los mejores resultados fueron obtenidos con una tasa de aprendizaje de 0.5, una profundidad máxima de 8 y 512 estimadores.

6.3 CNN

En el modelo de 1D-CNN, se utilizó el conjunto de datos de la extracción de características sin reducir la dimensionalidad. Asimismo, se ejecutó K-Fold, con el fin de saber

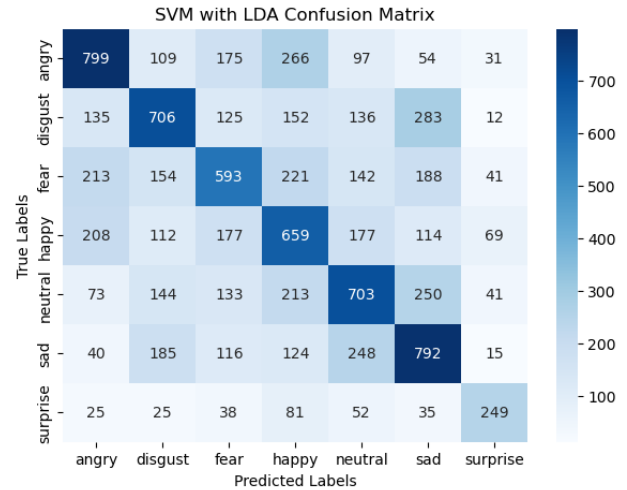


Figure 2. Matriz de confusión de clasificación con SVM con reducción LDA

tasa	prof. máx.	estimadores	test accuracy
1.0	8	256	0.78
1.0	8	512	0.79
1.0	10	256	0.77
1.0	10	512	0.78
0.5	8	256	0.82
0.5	8	512	0.83
0.5	10	256	0.81
0.5	10	512	0.82

Table 2. Búsqueda de hiperparámetros de XGBoost

que el modelo estaba generalizando de buena manera sin realizar *overfitting*. Cada aprendizaje con K-Fold duró 30 épocas.

Fold	accuracy	precision	f1-score
1	0.912	0.913	0.912
2	0.892	0.893	0.891
3	0.912	0.913	0.912
4	0.870	0.871	0.870
5	0.896	0.896	0.895

Table 3. K-Fold — Métricas de 1D-CNN

Ahora se presentará la matriz de confusión que haya arrojado los mejores valores en K-Fold

Por otro lado, se hizo una experimentación extra con el conjunto de datos sin **data augmentation** y sin K-Fold:

7. DISCUSIÓN

7.1 SVM

Como se pudo ver en la experimentación de SVM, el kernel RBF tiende a ser una opción más robusta para muchos problemas de clasificación, especialmente cuando las relaciones entre las características son complejas y no lineales. Esto resalta la importancia de elegir el kernel adecuado y ajustar los parámetros para evitar el overfitting y mejorar la generalización del modelo.

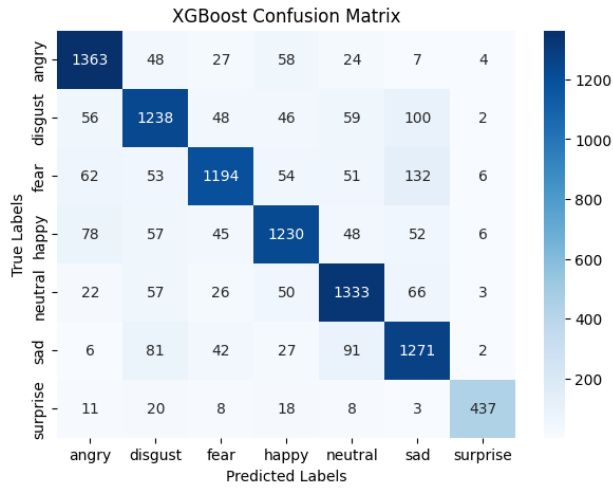


Figure 3. Matriz de confusión de clasificación con XG-Boost

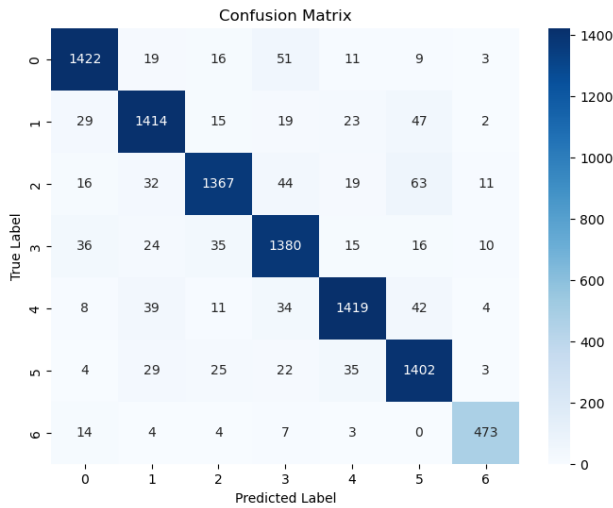


Figure 4. Matriz de confusión de clasificación 1D-CNN

7.2 XGBoost

En general, aumentar la cantidad de estimadores de XG-Boost siempre resulta en una mejora en *accuracy*. Por otro lado, aumentar la profundidad máxima representa una mejora sólo hasta cierto punto (profundidad máxima = 8); de ahí en adelante el desempeño del modelo empeora. Una posible explicación es que árboles muy profundos son más susceptibles a *overfitting*.

El mejor valor de *accuracy* para XGBoost fue de 0.83 en la data de testeo. El valor de *accuracy* en la data de entrenamiento para este modelo fue de 0.99. Esto es un indicador de que posiblemente se alcanzó un límite: sería difícil mejorar sustancialmente el modelo aumentando los estimadores o la profundidad, o cambiando la tasa de aprendizaje.

Asimismo, en base a la matriz de confusión, es posible calcular el valor de *accuracy* para cada clase:

Miedo es la emoción que es peor clasificada. En particular, el miedo (y también el disgusto) tiene una ligera tendencia a ser clasificado como tristeza, probablemente por la naturaleza negativa de estas emociones.

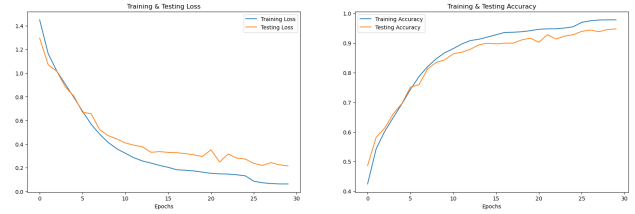


Figure 5. Curva de validación de 1D-CNN

	accuracy	precision	f1-score
test	0.558	0.572	0.552

Table 4. Métricas de 1D-CNN sin *data augmentation*

7.3 Convolutional Neural Networks

Con este modelo, se pudo notar que efectivamente las redes neuronales funcionan mucho mejor con una gran cantidad de datos, ya que si no se hubiera realizado *data augmentation*, el modelo se hubiera estancado en un "plató", tal cual se observa en la figura 6. En esta gráfica, se puede ver como aproximadamente a partir de la época siete el modelo empieza a hacer *overfitting*, dado que tanto la pérdida, como el *accuracy* empiezan a dejar de mejorar.

En el caso en el que se entrenó al modelo con el dataset con *data augmentation*, se puede ver una curva de validación muy buena, con menor riesgo de sobreajuste y una mejor generalización a nuevos datos. Esto indica que el aumento de datos ayudó a mejorar la robustez del modelo, permitiéndole aprender patrones más representativos en lugar de memorizar el conjunto de entrenamiento.

Como se mencionó en la sección de experimentación, el entrenamiento del modelo duró 30 épocas. Sin embargo, según la curva de validación, la curva de testeo aún no muestra signos de estancamiento, lo que sugiere que el modelo aún puede hacer uso de algunas épocas adicionales de entrenamiento para lograr un mejor ajuste a los datos y mayores métricas.

Asimismo, se halló el *accuracy* para cada emoción predicha:

Se observa que la emoción Surprise obtuvo la mejor precisión (0.94), mientras que "Fear" tuvo el desempeño más bajo (0.88) al igual que en XGB. Esto podría indicar que esta emoción es más compleja de diferenciarla, esto probablemente sea debido a su similitud con emociones como "Sad", esto lo confirmamos viendo la figura 5, donde en la intersección (5,2) es donde 63 etiquetas que debieron ser 2 (fear) se clasificaron como 5 (sad).

8. CONCLUSIONES

8.1 Resumen de resultados

- Realizar técnicas como *noise addition*, *time shifting* y *pitch shifting* incrementó la robustez de los modelos, evitando el sobreajuste y mejorando su generalización.
- La red 1D-CNN obtuvo el mejor desempeño, el *accuracy* es superior al 90 % en la clasificación de

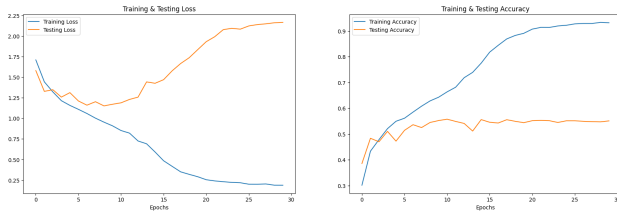


Figure 6. Curva de validación sin *data augmentation*

emoción	accuracy
Angry	0.89
Disgust	0.80
Fear	0.77
Happy	0.81
Neutral	0.86
Sad	0.84
Surprise	0.87

Table 5. Puntaje de accuracy de XGBoost por cada emoción.

emociones, superando significativamente a modelos tradicionales como SVM y XGBoost. Esto porque una CNN puede capturar patrones locales dentro de una señal, lo que le permite modelar relaciones más complejas y abstractas dentro de los datos, esto resulta en un modelo que generaliza mejor a nuevos inputs.

- Aunque no alcanzó la precisión de la CNN, XGBoost logró una precisión del 83 %, demostrando que los modelos basados en árboles de decisión pueden ser una alternativa viable en problemas de clasificación de audio. Esto es algo bueno, ya que este modelo tradicional requiere menos recursos de cómputo, por lo que podría ser entrenado en un ambiente con CPU o memoria limitada, igual obteniendo resultados bastante buenos.
- Tanto en SVM, XGBoost como en la CNN, la emoción *fear* mostró el peor *accuracy*, sugiriendo que esta emoción comparte similitudes con otras categorías como *sad* o *disgust*, lo que generó confusiones en la clasificación.
- El método de reducción de dimensionalidad LDA no tuvo muy buenos resultados: no fue capaz de separar los datos a pesar de tener las etiquetas. Esto sugiere que las clases no eran separables linealmente en primer lugar. Además, la reducción de 2376 a menos de 10 dimensiones probablemente hizo que se perdieran características relevantes para una buena distinción entre clases, lo cual afectó negativamente el desempeño del modelo de clasificación.

8.2 Limitaciones y recomendaciones

- Varios autores, además de analizar métricas por cada emoción, también analizan las métricas según el género de los actores de los audios. Como recomendación podría analizarse el desempeño con respecto

emoción	accuracy
Angry	0.92
Disgust	0.92
Fear	0.88
Happy	0.90
Neutral	0.93
Sad	0.92
Surprise	0.94

Table 6. Puntaje de accuracy de 1D-CNN por emoción.

al género, y así tal vez obtener posibles nuevos *insights*.

- El modelo fue entrenado con audios de alta calidad, por lo que su rendimiento en entornos ruidosos o con variaciones en la grabación fueron evaluados de manera limitada. Si bien se agregó ruido a la data, esto fue de manera sintética y podría no reflejar la realidad. Como recomendación se debe entrenar el modelo con audios de diferentes calidades y ambientes para tener un entrenamiento mucho más real.
- Algunas emociones tenían más muestras que otras, afectando la equidad en el aprendizaje del modelo. Como recomendación se encuentra la implementación de técnicas como oversampling o undersampling para poder equilibrar clases y mejorar la generalización del modelo.
- El proceso de entrenamiento de la CNN fue lenta y costosa computacionalmente debido a que inicialmente se usó solo la CPU, sin embargo se optimizó el rendimiento utilizando una tarjeta gráfica *Nvidia GeForce RTX 4060 con 8GB de VRam* lo cual redujo significativamente el tiempo de entrenamiento. En futuras implementaciones se recomendaría el uso de GPU en cloud para poder utilizar tarjetas gráficas aún más potentes para acelerar el entrenamiento.

9. REFERENCIAS

- [1] M. Jain et al., "Speech Emotion Recognition using Support Vector Machine," arXiv:2002.07590 [cs, eess], Feb. 2020, Available: <https://arxiv.org/abs/2002.07590>
- [2] Y. Pan, P. Shen, and L. Shen, "Speech Emotion Recognition Using Support Vector Machine," International Journal of Smart Home, vol. 6, no. 2, 2012, Accessed: Feb. 22, 2025. [Online]. Available: https://gvpress.com/journals/IJSH/vol6_no2/15.pdf
- [3] Houwei Cao, Ragini Verma, Ani Nenkova, "Speaker-sensitive emotion recognition via ranking: Studies on acted and spontaneous speech, Computer Speech & Language", Volume 29, Issue 1, 2015, Pages 186-202, ISSN 0885-2308. Available: <https://doi.org/10.1016/j.csl.2014.01.003>.
- [4] M. Gokilavani, H. Katakam, S. A. Basheer and P. Srinivas, Ravdness, Crema-D, Tess Based Algorithm for Emotion Recognition Using Speech,"2022 4th International Conference on Smart Systems and Inventive Techno-

logy (ICSSIT), Tirunelveli, India, 2022, pp. 1625-1631, doi: 10.1109/ICSSIT53264.2022.9716313.

[5] B. T. Atmaja, K. Shirai and M. Akagi, "Speech Emotion Recognition Using Speech Feature and Word Embedding,"2019 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), Lanzhou, China, 2019, pp. 519-523, doi: 10.1109/APSIPAASC47483.2019.9023098.

[6] Singh J, Saheer LB, Faust O. Speech Emotion Recognition Using Attention Model. Int J Environ Res Public Health. 2023 Mar 14;20(6):5140. doi: 10.3390/ijerph20065140. PMID: 36982048; PMCID: PMC10049636.

[7] C. M. Bishop, Pattern Recognition and Machine Learning. Springer, 2006.

10. CONTRIBUCIÓN

	Andrea	Nayeli	Luis	Matías
Introducción		X		
Revisión de la literatura	X			
Dataset		X		
Metodología	X	X	X	X
Implementación	X	X	X	X
Experimentación	X	X	X	X
Discusión	X	X	X	X
Conclusiones	X		X	

Table 7. Contribución en la redacción

11. APÉNDICE

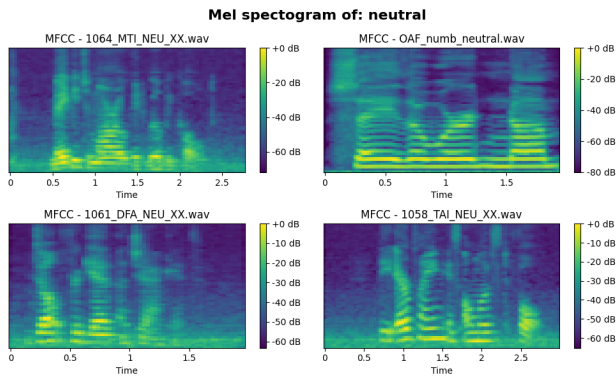


Figure 7. Espectrograma de Mel de 'Neutral'

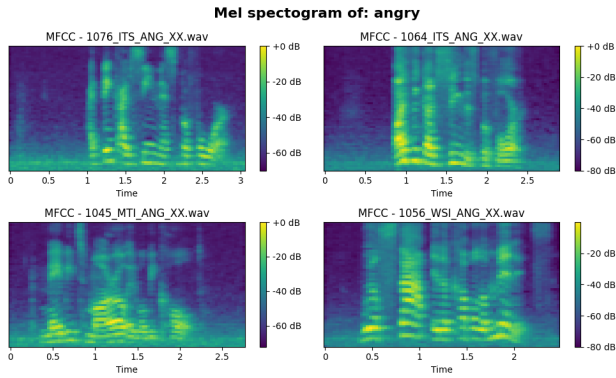


Figure 8. Espectrograma de Mel de 'Angry'

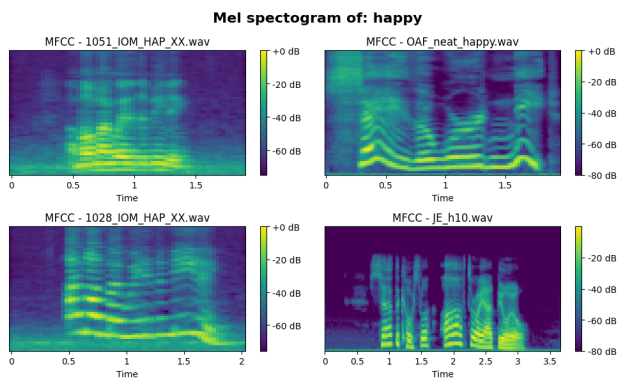


Figure 9. Espectograma de Mel de 'Happy'